

Apache Kafka is an open-source distributed event streaming platform used to build realtime data pipelines and streaming applications.

It is designed to handle high-throughput, low-latency, fault-tolerant, and scalable messaging. Simple Definition: Kafka is like a high-performance message broker that lets different systems communicate by publishing and subscribing to streams of records (messages/events), similar to a distributed log system.

Top Reasons We Need Kafka

1. Decoupling Services

In microservices architecture, services need to communicate. Kafka enables:

- One service to **publish data/events**
- Other services to **consume** them **independently**, without tight coupling **Example:** When a user places an order:
- order-service sends the order data to Kafka
- payment-service, inventory-service, and notification-service independently consume the message and act accordingly

2. Handling Real-Time Data Streams

Kafka is built for **real-time data pipelines**:

- It can process **events** as they happen (e.g., clicks, transactions, sensor data)
- Enables **real-time analytics**, alerts, dashboards, etc. **Example:** Fraud detection system that checks transactions as they happen

3. High Throughput and Scalability

Kafka is designed to handle:

- **Millions of messages per second**
- **Horizontal scaling** by adding more brokers and partitions Good for **big data** and high-traffic systems (e.g., LinkedIn, Uber)

4. Reliable & Durable Messaging

Kafka:

- Stores messages **on disk** and keeps them for configurable periods

- Supports **replication**, so no data is lost if a broker fails Unlike in-memory queues, Kafka is **durable** and **resilient**

5. Event Sourcing and Audit Logs

Kafka maintains an **ordered, immutable log** of all events:

- Systems can **replay** events (like a time machine)
- Ideal for **audit trails** or **event sourcing** architectures Example: A service can re-process all orders if business logic changes

6. Integration Across Systems

Kafka acts as a **central hub** that connects different systems, databases, APIs, and services. You can connect Kafka with:

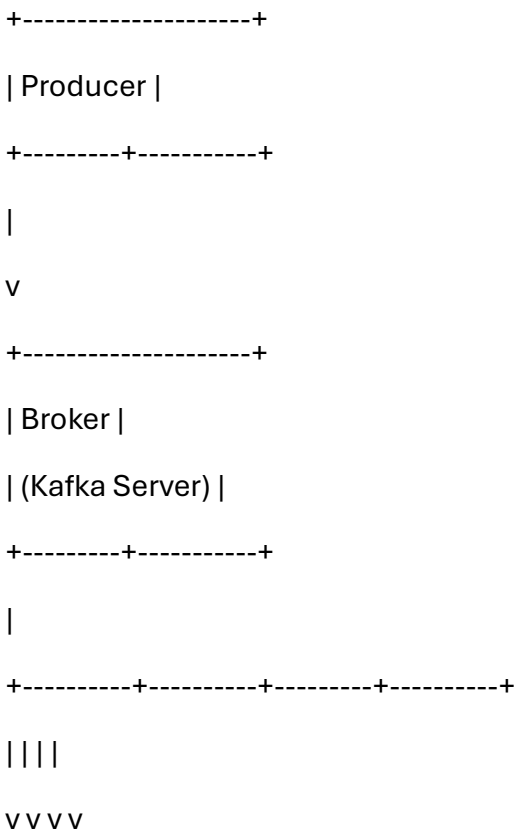
- Relational DBs
- NoSQL DBs
- Data lakes (via Kafka Connect)
- Stream processors like Flink or Spark

Kafka Core Components

Component	Description
--	
Producer	Sends (publishes) messages to Kafka topics.
Consumer	Reads messages from topics.
Broker	Kafka server that stores and serves messages.
Topic	Named stream where records are sent; like a message category.
Partition	Subdivision of a topic; enables parallelism and scaling.

Component	Description
Offset	Position (ID) of a message within a partition.
Consumer Group	A group of consumers that divides the work of consuming a
topic.	
Zookeeper (deprecated)	Used for coordination; Kafka is replacing it with **KRaft
(Kafka Raft Metadata mode)**.	

Kafka Internal Architecture (Simplified View)



Partition-0	Partition-1	Partition-2	Partition-N
(Topic-A)	(Topic-A)	(Topic-A)	(Topic-A)
v v v v			
Offset: 0..n	Offset: 0..n	Offset: 0..n	Offset: 0..n
^			
v			
+-----+-----+	+-----+-----+		
Consumer (Group A)	Consumer (Group B)		
+-----+	+-----+		

Internal Working of Kafka

1. Topics and Partitions

- A **topic** is divided into **partitions**.
- Each partition is an **append-only log**.
- Each message is assigned a sequential **offset**.

2. Broker

- Kafka runs as a **cluster of brokers**.
- Each broker hosts a subset of partitions.
- A broker that manages a partition's read/write is called the **leader** for that partition; others are **replicas**.

3. Producer

- Sends messages to a topic.
- Can choose:
 - Specific partition
 - Let Kafka decide using a partitioning strategy (e.g., round-robin or key-based)

4. Consumer

- Reads from one or more partitions.
- Tracks the **offset** of consumed messages.
- In **consumer groups**, Kafka ensures that **each partition is read by only one consumer** in the group.

5. Replication

- Each partition can have **multiple replicas**.
- Kafka ensures:
 - One **leader**
 - One or more **followers**
- Replication ensures **fault tolerance**

6. Retention

- Kafka retains messages for a **configurable period** (e.g., 7 days).
- Consumers can re-read messages from any point using the **offset**.

7. Zookeeper / KRaft

- Kafka uses **Zookeeper** (in older versions) for:
 - Broker coordination
 - Leader election
 - Topic metadata management
- **KRaft Mode** is Kafka's built-in replacement for Zookeeper (available from Kafka 2.8+)

Kafka Cluster Example:

- **3 Brokers**
- **1 Topic** with 4 partitions and replication factor 2

Partition	Leader	Replica 1	Replica 2
p0	Broker 1	Broker 2	Broker 3
p1	Broker 2	Broker 3	Broker 1
p2	Broker 3	Broker 1	Broker 2
p3	Broker 1	Broker 2	Broker 3

2. Kafka Configuration (application.yml)

spring:

kafka:

bootstrap-servers: localhost:9092

consumer:

group-id: my-group

auto-offset-reset: earliest

key-deserializer: org.apache.kafka.common.serialization.StringDeserializer

value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

producer:

key-serializer: org.apache.kafka.common.serialization.StringSerializer

value-serializer: org.apache.kafka.common.serialization.StringSerializer

3. Kafka Producer

```
import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.kafka.core.KafkaTemplate; import
org.springframework.stereotype.Service; @Service public class KafkaProducer { private
static final String TOPIC = "test-topic"; @Autowired private KafkaTemplate<String, String>
kafkaTemplate; public void sendMessage(String message) { kafkaTemplate.send(TOPIC,
message); System.out.println("Sent message: " + message); }}
```

4. Kafka Consumer

```
import org.springframework.kafka.annotation.KafkaListener; import
org.springframework.stereotype.Service; @Service public class KafkaConsumer {
@KafkaListener(topics = "test-topic", groupId = "my-group") public void consume(String
message) { System.out.println("Received message: " + message); }}
```

Sending a Message to a Specific Partition

java

Copy

Edit

@Service

```
public class CustomPartitionProducer {
```

```
@Autowired
```

```
private KafkaTemplate<String, String> kafkaTemplate;
```

```
public ListenableFuture<SendResult<String, String>> sendToPartition(String topic, int
partition, String key, String message) {
```

```
return kafkaTemplate.send(topic, partition, key, message);
```

```
}
```

```
}
```

****Apache Kafka custom object serialization & deserialization example**** using ****Spring Boot****, including:

1. Custom Java object (POJO)
2. JSON serialization/deserialization
3. Kafka producer & consumer using object messages
4. Optional: Registering custom (de)serializers (for non-JSON)

1. **Add Dependencies (`pom.xml`)**

```
` `` `xml
```

```
<dependencies>

<!-- Spring Boot Web & Kafka -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.kafka</groupId>

<artifactId>spring-kafka</artifactId>

</dependency>

<!-- JSON support -->

<dependency>

<groupId>com.fasterxml.jackson.core</groupId>

<artifactId>jackson-databind</artifactId>

</dependency>

</dependencies>
```

2. Create a Java Object (POJO)

```
public class User {

    private String id;

    private String name;

    private int age;

    // Constructors

    public User() {}

    public User(String id, String name, int age) {

        this.id = id;
```



```
this.name = name;

this.age = age;

}

// Getters & Setters

public String getId() { return id; }

public String getName() { return name; }

public int getAge() { return age; }

public void setId(String id) { this.id = id; }

public void setName(String name) { this.name = name; }

public void setAge(int age) { this.age = age; }

}
```

3. Kafka Configuration with JSON Serializer/Deserializer

@Configuration

```
public class KafkaConfig {
```

@Bean

```
public ProducerFactory<String, User> producerFactory() {
```

```
    Map<String, Object> config = new HashMap<>();
```

```
    config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
```

```
    config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
```

```
    config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
```

```
    return new DefaultKafkaProducerFactory<>(config);
```

```
}
```

@Bean

```
public KafkaTemplate<String, User> kafkaTemplate() {
```

```
    return new KafkaTemplate<>(producerFactory());
```

```

}

@Bean

public ConsumerFactory<String, User> consumerFactory() {
    Map<String, Object> config = new HashMap<>();
    config.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
    config.put(ConsumerConfig.GROUP_ID_CONFIG, "user-group");
    config.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
    config.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
    config.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
JsonDeserializer.class);
    config.put(JsonDeserializer.TRUSTED_PACKAGES, "*");
    return new DefaultKafkaConsumerFactory<>(
        config, new StringDeserializer(), new JsonDeserializer<>(User.class)
    );
}

@Bean

public ConcurrentKafkaListenerContainerFactory<String, User>
kafkaListenerContainerFactory() {
    ConcurrentKafkaListenerContainerFactory<String, User> factory =
        new ConcurrentKafkaListenerContainerFactory<>();
    factory.setConsumerFactory(consumerFactory());
    return factory;
}
}

```

4. Producer Example (Send User Object)

```
@Service

public class UserProducer {

    @Autowired

    private KafkaTemplate<String, User> kafkaTemplate;

    private final String topic = "user-topic";

    public void sendUser(User user) {

        kafkaTemplate.send(topic, user);

        System.out.println("Sent user: " + user.getName());

    }

}
```

5. Consumer Example (Receive User Object)

```
@Service

public class UserConsumer {

    @KafkaListener(topics = "user-topic", groupId = "user-group", containerFactory =
    "kafkaListenerContainerFactory")

    public void consume(User user) {

        System.out.println("Received User -> ID: " + user.getId() + ", Name: " + user.getName());

    }

}
```

Message Routing with Specific Partitions in Apache Kafka

Kafka allows you to route messages to specific partitions to ensure order, load balancing, or custom logic (e.g., key-based grouping).

3 Ways to Route Messages to Partitions

1. Default Routing (Round-Robin or Key-Based)

Kafka uses:

- Round-robin if **no key** is provided
- **Key-based hashing** if a key is set: $\text{partition} = \text{hash}(\text{key}) \% \text{numPartitions}$

```
kafkaTemplate.send("topic-name", "key1", "value1");
```

Benefit: Keeps all messages with same key on same partition.

2. Manually Target a Partition (Direct Routing)

```
kafkaTemplate.send("my-topic", 2, "my-key", "message for partition 2");
```

Parameter	Meaning
"my-topic"	Topic name
2	Partition number (manual)
"my-key"	Message key
"value"	Message body
Note: Ensure the topic has at least 3 partitions.	

3. Custom Partitioning Logic (Custom Partitioner Class)

A. Implement Your Own Partitioner:

```
public class CustomPartitioner implements Partitioner {  
    @Override  
    public int partition(String topic, Object keyObj, byte[] keyBytes,  
        Object value, byte[] valueBytes, Cluster cluster) {  
        String key = (String) keyObj;  
        // Example: Route keys starting with "A" to partition 0  
        if (key.startsWith("A")) return 0;  
    }  
}
```

```

if (key.startsWith("B")) return 1;

return 2;

}

@Override public void close() {}

@Override public void configure(Map<String, ?> configs) {}

}

```

B. Register the Partitioner in Configuration:

```

@Bean

public ProducerFactory<String, String> producerFactory() {

    Map<String, Object> config = new HashMap<>();

    config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");

    config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);

    config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);

    config.put(ProducerConfig.PARTITIONER_CLASS_CONFIG, CustomPartitioner.class);

    return new DefaultKafkaProducerFactory<>(config);

}

```

Then, use as usual:

```
kafkaTemplate.send("my-topic", "A123", "value with custom partitioning");
```

How to Confirm the Partition Routing?

You can inspect the `SendResult` returned by `KafkaTemplate`:

```

kafkaTemplate.send("my-topic", "key", "value").addCallback(result -> {

    if (result != null) {

        System.out.println("Message sent to partition: " +

            result.getRecordMetadata().partition());

    }

}

```

```
}, ex -> {  
  
    System.err.println("Send failed: " + ex.getMessage());  
  
});
```

Consumer Side Consideration

@KafkaListener(topics = "javatechie-topic", groupId = "jt-group", topicPartitions =
@TopicPartition(topic = "javatechie-topic", partitions = {"2"})) public void
consumeEvents(String customer) { log.info("consumer consume the events {}",
customer.toString()); } This setup means: You're listening only to partition 2 of javatechie-
topic. Kafka will not auto-assign other partitions to this consumer. It's manual static
assignment, not dynamic partition balancing.

Kafka Retry and Dead-Letter Logic – Background

In **traditional message consumption**, when an exception occurs, either:

- The consumer **keeps retrying** (blocking the thread), or
 - The message is **skipped/lost**, or
 - You must implement a **custom retry mechanism**. Spring Kafka solves this with @RetryableTopic, which gives **non-blocking**, topicbased retry handling + automatic DLT.
-

What Happens When You Use @RetryableTopic

```
@RetryableTopic(  
    attempts = "3",  
    backoff = @Backoff(delay = 3000, multiplier = 2, maxDelay = 15000)  
)  
  
@KafkaListener(topics = "order-events", groupId = "order-group")  
public void consumeEvents(Order order) {  
    // Processing logic  
}
```

✂ Behind the Scenes:

1. **Topic order-events is your main topic.**
2. Spring Kafka **creates retry topics**:
 - order-events-retry-0
 - order-events-retry-1 **Retry Topics = attempts - 1**, so 3 attempts → 2 retry topics.
3. When an exception occurs in the original listener:
 - The message is **not retried in-place**.
 - It is re-published to the **next retry topic**.
4. Each retry topic has:
 - Its own delay (based on backoff strategy)
 - Its own Kafka listener internally registered
5. After exhausting all retries:
 - Message is sent to a **Dead Letter Topic**:
 - order-events-dlt

Example Timeline

Let's say:

- attempts = 4
- delay = 3000, multiplier = 1.5
- Failures happen on every attempt

Attempt #	Topic	Delay (ms)	Listener invoked
1	order-events	0	Original
2	order-events-retry-0	3000	Retry #1

Attempt #	Topic	Delay (ms)	Listener invoked
3	order-events-retry-1	4500	Retry #2
4	order-events-retry-2	6750	Retry #3
—	order-events-dlt	—	Dead Letter
Total retry attempts = attempts - 1			
DLT only happens if all retries fail			

Real-World Notes:

- **Retry topics are regular Kafka topics** — persisted and visible in your Kafka cluster.
 - You can customize their name with `dltTopicSuffix`, `retryTopicSuffix`.
 - You must **avoid rethrowing fatal exceptions** unless you want them to skip retry and go directly to DLT (use `exclude = {}`).
 - You can enable/disable `autoCreateTopics` in production.
-

Handling the DLT

```
@KafkaListener(topics = "order-events-dlt", groupId = "dlt-handler")

public void processFailedMessages(Order failedOrder) {

    log.warn("Dead-lettered order: {}", failedOrder);

    // Save to DB, notify, etc.

}
```

Summary

Concept	Meaning
@RetryableTopic	Declarative retry config for Kafka listeners
attempts	Total tries (original + retry topics), creates attempts - 1 topics
backoff	Delay strategy for retry topic processing
Retry topic naming	topic-retry-0, topic-retry-1, ...
DLT topic	topic-dlt, receives message after final failure
exclude/include	Controls which exceptions are retried vs sent directly to DLT

Here's a complete, **practical, production-ready example** of using **Kafka with Schema Registry and Avro** in **Spring Boot**, including:

- What Avro & Schema Registry do
 - Why and when to use them
 - Spring Boot integration
 - Producer & consumer with Avro
 - Setup via Docker
-

What Are Avro & Schema Registry?

Concept	Description
--	

Concept	Description
Avro	A compact, fast binary serialization format (better than JSON)
Schema Registry	A centralized service to manage Avro schemas (used during
serialization & deserialization)	
Why?	Prevent schema mismatches, reduce payload size, version-safe changes

Use Case Example

Let's say we're sending **User** objects over Kafka.

```
{
  "id": "u123",
  "name": "Alice",
  "email": "alice@example.com"
}
```

Using Avro + Schema Registry helps you:

- Define this structure via .avsc file (schema)
- Validate producer and consumer are in sync
- Store schema versions centrally

Setup: Kafka + Schema Registry with Docker

docker-compose.yml

version: '2'

services:

zookeeper:

image: confluentinc/cp-zookeeper:7.5.0

environment:

ZOOKEEPER_CLIENT_PORT: 2181

kafka:

image: confluentinc/cp-kafka:7.5.0

depends_on: [zookeeper]

ports:

- "9092:9092"

environment:

KAFKA_BROKER_ID: 1

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

schema-registry:

image: confluentinc/cp-schema-registry:7.5.0

depends_on: [kafka]

ports:

- "8081:8081"

environment:

SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: PLAINTEXT://kafka:9092

SCHEMA_REGISTRY_HOST_NAME: schema-registry

Run:

docker-compose up -d

Define Avro Schema (User)

src/main/avro/user.avsc

```
{  
  "type": "record",  
  "name": "User",  
  "namespace": "com.example.avro",  
  "fields": [  
    {"name": "id", "type": "string"},  
    {"name": "name", "type": "string"},  
    {"name": "email", "type": "string"}  
  ]  
}
```

Add Maven Dependencies

pom.xml

```
<dependencies>  
  <!-- Kafka Avro Serializer -->  
  <dependency>  
    <groupId>io.confluent</groupId>  
    <artifactId>kafka-avro-serializer</artifactId>  
    <version>7.5.0</version>  
  </dependency>  
  <!-- Spring Kafka -->  
  <dependency>  
    <groupId>org.springframework.kafka</groupId>
```

```
<artifactId>spring-kafka</artifactId>

</dependency>

<!-- Avro Maven Plugin -->

<plugin>

<groupId>org.apache.avro</groupId>

<artifactId>avro-maven-plugin</artifactId>

<version>1.11.1</version>

<executions>

<execution>

<phase>generate-sources</phase>

<goals><goal>schema</goal></goals>

<configuration>

<sourceDirectory>${project.basedir}/src/main/avro</sourceDirectory>

<outputDirectory>${project.basedir}/src/main/java</outputDirectory>

</configuration>

</execution>

</executions>

</plugin>

</dependencies>
```

Run this to generate Java class from Avro schema:

```
mvn clean compile
```

Generates: com.example.avro.User.java

Kafka Configuration (Avro + Schema Registry)

KafkaConfig.java

```
@Configuration
```

```

public class KafkaConfig {

    @Value("${spring.kafka.bootstrap-servers}")
    private String bootstrapServers;

    @Value("${spring.kafka.schema-registry-url}")
    private String schemaRegistryUrl;

    @Bean

    public ProducerFactory<String, User> producerFactory() {
        Map<String, Object> config = new HashMap<>();
        config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
KafkaAvroSerializer.class);
        config.put("schema.registry.url", schemaRegistryUrl);
        return new DefaultKafkaProducerFactory<>(config);
    }

    @Bean

    public KafkaTemplate<String, User> kafkaTemplate() {
        return new KafkaTemplate<>(producerFactory());
    }

    @Bean

    public ConsumerFactory<String, User> consumerFactory() {
        Map<String, Object> config = new HashMap<>();
        config.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        config.put(ConsumerConfig.GROUP_ID_CONFIG, "avro-group");
        config.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);
    }
}

```

```

config.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
KafkaAvroDeserializer.class);
config.put("schema.registry.url", schemaRegistryUrl);
config.put(KafkaAvroDeserializerConfig.SPECIFIC_AVRO_READER_CONFIG, true);
return new DefaultKafkaConsumerFactory<>(config);
}

@Bean

public ConcurrentKafkaListenerContainerFactory<String, User>
kafkaListenerContainerFactory() {
    var factory = new ConcurrentKafkaListenerContainerFactory<String, User>();
    factory.setConsumerFactory(consumerFactory());
    return factory;
}
}

```

Producer Code

```

@Service

public class UserProducer {

    @Autowired

    private KafkaTemplate<String, User> kafkaTemplate;

    public void send(User user) {
        kafkaTemplate.send("user-avro-topic", user);

        System.out.println("Produced Avro user: " + user.getName());
    }
}

```

Consumer Code

```
@Service

public class UserConsumer {

    @KafkaListener(topics = "user-avro-topic", groupId = "avro-group", containerFactory =
"kafkaListenerContainerFactory")

    public void consume(User user) {

        System.out.println("Consumed Avro user: " + user.getName());

    }

}
```

Test via Controller

```
@RestController

@RequestMapping("/user")

public class UserController {

    @Autowired

    private UserProducer producer;

    @PostMapping("/publish")

    public String sendUser(@RequestBody Map<String, String> payload) {

        User user = new User(payload.get("id"), payload.get("name"), payload.get("email"));

        producer.send(user);

        return "Avro User Sent";

    }

}
```

Test the Setup

1. Start Kafka + Schema Registry via Docker

2. Run the Spring Boot app

3. POST to:

POST http://localhost:8080/user/publish

Content-Type: application/json

```
{  
  "id": "u1",  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

You should see:

- Avro object serialized
- Schema auto-registered in Schema Registry
- Consumed properly

Summary

Component	Purpose
Avro Schema	Defines structured messages
Schema Registry	Stores & manages schema versions
KafkaAvroSerializer	Converts POJO to Avro bytes
KafkaAvroDeserializer	Converts Avro bytes back to POJO
Spring Kafka	Integrates Kafka Avro messaging in Spring

Great question — and your interest in going beyond the basics is exactly what leads to true Kafka mastery. Here’s a breakdown of important Kafka topics and concepts that people commonly skip, but are critical for real-world, production-grade usage.

High-Value Kafka Topics You Shouldn't Skip

1. Kafka Security (ACLs, SSL, SASL, Authorization)

| What You Might Miss | Why It Matters

|

| ----- | -----
----- || Kafka supports **encryption, authentication,**
and **authorization** via SSL, SASL, and ACLs | Needed in production, especially in regulated
environments | | Common configs: security.protocol, ssl.keystore, sasl.mechanism |
Without this, your Kafka cluster is wide open |

Also: Role-based access control using tools like Confluent RBAC

2. Kafka Internals: Partitions, Offsets, Replication, ISR

Topic	Why It’s Important
Partition	Enables parallelism & scalability
Offset	Consumers track where they are in the topic
Replication	Guarantees durability (n+1 broker failure tolerance)
In-Sync Replicas (ISR)	Determines safe leaders for failover
Must know: What happens when a broker dies? How does leadership change?	

3. Kafka Exactly-Once Semantics (EOS)

- Often skipped because it's **harder to configure**
- Prevents **duplicate events**, useful in:
- Payments
- Stock trades
- Inventory processing ✂ Involves:
- Idempotent producer (enable.idempotence=true)
- Transactions (initTransactions(), beginTransaction()...)

4. Consumer Rebalancing Strategies & Sticky Assignments

- Most people know group rebalancing **happens**, but not **how to control it**
- Custom strategies (like **StickyAssignor**) can reduce overhead
- Use KafkaRebalanceListener in Spring for state-safe rebalancing

5. Kafka Streams & ksqlDB

Tool	Use Case
Kafka Streams	Java API for streaming joins, aggregations
ksqlDB	SQL over Kafka topics (no code!)
Use this for:	

- Enrichment
- Real-time dashboards
- Data validation
- Event correlation

6. Retention, Compaction, and Cleanup Policies

Topic Property	What it does
retention.ms	Deletes messages after time
retention.bytes	Deletes after size exceeds limit
cleanup.policy=compact	Keeps latest record per key
Example: For user profiles, you may want only the latest state per user , not every	
update — use log compaction .	

7. Kafka Topic Design Best Practices

Design Pattern	Benefit
One topic per event type	Simple schema evolution
Use consistent keys	Ensures partition affinity
Avoid large message payloads	Reduces memory pressure
Partition count based on consumers	Maximizes parallelism

8. Dead Letter Topics (DLT) & Retry Strategies

- How to design retry topics (-retry, -dlt)
- Manual vs @RetryableTopic in Spring
- Handling poison messages safely Must understand difference between:
- Blocking retry

- Topic-based retry
 - Error handling vs skip logic
-

9. Kafka in Cloud: Confluent Cloud / AWS MSK

Cloud Kafka	Notes
AWS MSK	Fully managed Kafka, supports IAM auth, integrates with VPC
Confluent Cloud	SaaS Kafka + Schema Registry, REST Proxy, etc.
Learn:	

- How to connect securely
 - How to monitor usage
 - Limits vs self-managed
-

10. Monitoring, Metrics & Kafka Lag

Tool	Purpose
Prometheus + Grafana	Broker/Consumer monitoring
Burrow, Kafka Lag Exporter	Monitor consumer group lag
JMX metrics	Deep Kafka internals
Monitor:	

- Consumer group lag
 - Under-replicated partitions
 - Broker memory, disk, CPU
-

